

19 · Matriz de trazabilidad

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 19-traceability-matrix.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

Cómo leer estas tablas

1. Los 27 casos operativos
2. Capacidades del motor
3. Capacidades del panel
4. Reglas de negocio verificables
5. De la persistencia hacia atrás
6. Resumen de cobertura de trazabilidad

Cada fila permite seguir una funcionalidad **en una sola línea**, desde la interfaz hasta la persistencia y sus pruebas. Los datos proceden de la lectura de los 27 manifests, del registro de acciones y de la ejecución de la suite en el commit **ff246ab**.

Cómo leer estas tablas

Columna	Significado
Interfaz	Cómo se dispara: tarjeta del panel, ruta HTTP, comando
Manifest	La receta que la implementa
Pasos → acciones	La cadena de acciones, en orden
Módulo · función	El código Python que hace el trabajo
Persistencia	Dónde acaban los datos
Prueba	El archivo de pruebas que la cubre, o el hueco
Doc	Dónde se explica
Estado	[OK] verificado · [!] parcial · [NO] sin cobertura · [manual] requiere validación manual

1. Los 27 casos operativos

1.1 Familia **pantalla** — 6 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-01	Capturar el escritorio y analizar brillo/RGB	Tarjeta · Alt+1 · POST /api/run/01_screen_capture_analyze	01_screen_capture_analyze	capture_screen→screen.capture_screen; analyze_capture→vision.analyze_image; write_report→filesystem.write_json	actions/screen.py::capture_screenshot; plugins/analyzers/mock_image_analyzer.py::MockImageAnalyzer; actions/filesystem.py::write_json	output/screenshots/*.png, output/reports/*.json, runs, steps, events	[NO] Sin prueba de la captura real	05 §1.2, 06 §13	[manual]
F-09	Minimizar todo y capturar escritorio limpio	Tarjeta · POST /api/run/09_show_desktop_capture	09_show_desktop_capture	minimize_all_windows→ui.hotkey; wait_animation→system.wait_seconds; capture_desktop→screen.capture_screenshots	actions/ui.py::hotkey; actions/system.py::wait_seconds; actions/screen.py::capture_screenshot	output/screenshots/desktop_clean*.png + histórico	[!] test_actions_basic.py cubre hotkey en dry_run	05 §1.5	[manual]
F-12	Inventario OCR de todo el texto visible	Tarjeta · POST /api/run/12_desktop_ocr_inventory	12_desktop_ocr_inventory	capture_desktop→screen.capture_screenshot; ocr_full_image→vision.ocr_image; save_inventory→filesystem.write_json	actions/vision.py::ocr_image; plugins/analyzers/ocr_image_analyzer.py::OCRImageAnalyzer	output/screenshots/, output/reports/ + histórico	[NO] actions/vision.py al 0%; plugins/fuera del ámbito de cobertura	09 §4.5, 06 §13	[manual]
F-16	Captura solo de la ventana en foco	Tarjeta · POST /api/run/16_active_window_screenshot	16_active_window_screenshot	capture_active_window→screen.capture_active_window	actions/screen.py::capture_active_window + pygetwindow	output/screenshots/*.png + histórico	[NO] Requiere ventana con foco	05 §1.2	[manual]

F-17	Snapshot del Administrador de tareas con OCR	Tarjeta · <code>POST /api/run/17_taskmgr_snapshot</code>	17_taskmgr_snapshot	<code>ui.hotkey; system.wait_seconds; screen.capture_screenshot; vision.ocr_image; filesystem.write_json</code> (5 pasos)	Los cuatro módulos anteriores	<code>output/screenshots/output/reports/</code> + histórico	[NO]	04 \$5	[manual]
F-19	Captura de la barra de tareas	Tarjeta · <code>POST /api/run/19_taskbar_capture</code>	19_taskbar_capture	<code>capture_taskbar→screen.capture_region</code> con <code>bbox</code> de <code>top: -48</code>	<code>actions/screen.py::capture_region, resolve_bbox</code>	<code>output/screenshots/taskbar_*.png</code> + histórico	[NO]	05 \$1.2	[manual]

1.2 Familia navegador — 9 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-02	Capturar el DOM renderizado de una URL	Tarjeta · <code>Alt+2</code> (pide URL)	02_screen_capture_browser	<code>browser.capture_page</code>	<code>actions/browser_capture.py::capture_page, to_url</code>	<code>output/screenshots/*.png</code> + histórico	[!] 18 % : solo <code>to_url</code>	09 \$4.1	[manual]
F-07	Rellenar un formulario de 10 campos sin repetir registro	Tarjeta · <code>Alt+7</code>	07_browser_form_filler	<code>fill_and_submit_form→browser.fill_form</code>	<code>actions/browser_form.py::fill_form, pick_record, save_used_ids</code>	<code>output/reports/form_submission_*.json</code> ; tracking en data/seeds/.used_indices.json	[NO] 0 %	05 \$1.8, 06 \$8, 08 \$10.3	[manual]
F-21	Extraer título, texto, links, metadatos y tablas	Tarjeta · <code>POST /api/run/21_web_content_extract</code>	21_web_content_extract	<code>browser.extract_content; browser.capture_page; filesystem.write_json</code>	<code>actions/browser_extract.py::extract_content, build_result, resolve_links, normalize_text, content_hash</code>	<code>output/reports/*.json</code> , PNG opcional + histórico	[OK] <code>test_browser_extract.py</code> — 91 % de la lógica pura	05 \$1.8, 06 \$7	[!]
F-22	Mapa de sitio con BFS acotado y robots.txt	Tarjeta · <code>POST /api/run/22_web_site_map</code>	22_web_site_map	<code>browser.crawl_site; filesystem.write_json</code>	<code>actions/browser_extract.py::crawl_site, crawl_pages, RobotsCache</code>	<code>output/reports/*.json</code> + histórico	[OK] <code>test_browser_extract.py</code> con <code>fetch_page</code> falso	06 \$7.5–7.7	[!]
F-23	Detectar cambios en una página y notificar	Tarjeta · <code>POST /api/run/23_web_change_detector</code>	23_web_change_detector	<code>extract_and_track→browser.extract_content</code> (retries: 1); <code>evaluate_change→rules.evaluate; notify_change→notify.send condicionado; write_report→filesystem.write_json</code>	<code>browser_extract.py::apply_tracking, content_hash; actions/rules.py::evaluate_rules; actions/notify.py::send_notification</code>	<code>output/reports/, output/notifications/web_changes.log</code> ; tracking en data/web_watch/demo_page.json	[OK] <code>apply_tracking</code> y <code>content_hash</code> cubiertos; [!] el <code>when</code> del paso 3, en <code>test_orchestrator.py</code>	08 \$10.2, 14 \$5.3	[!]
F-24	Auditar enlaces rotos de una página	Tarjeta · <code>POST /api/run/24_web_link_audit</code>	24_web_link_audit	<code>extract_links→browser.extract_content; check_links→http.check_urls; evaluate_links→rules.evaluate; write_report→filesystem.write_json</code>	<code>browser_extract.py::resolve_links; actions/http_actions.py::check_urls</code>	<code>output/reports/link_audit_*.json</code> + histórico	[OK] <code>resolve_links</code> y <code>check_urls</code> en <code>test_browser_extract.py</code>	05 \$1.6	[!]
F-25	Extraer cada <code><table></code> a CSV y JSON	Tarjeta · <code>POST /api/run/25_web_table_extract</code>	25_web_table_extract	<code>browser.extract_content</code> con <code>include_tables</code> ; <code>filesystem.write_json</code>	<code>browser_extract.py::save_tables_csv, build_result</code>	<code>output/reports/*.json</code> , <code>table_NN.csv</code> + histórico	[OK] <code>save_tables_csv</code> cubierto	05 \$1.8	[!]

F-26	Vigilar un valor por selector CSS y alertar	Tarjeta · <code>POST /api/run/26_web_value_monitor</code>	<code>26_web_value_monitor</code>	<code>read_value→browser.extract_content(selector #precio); evaluate_value→rules.evaluate; notify_value→notify.send condicionado; write_report→filesystem.write_json</code>	<code>browser.extract.py::parse_number, appily_tracking</code>	<code>output/reports/, output/notificaciones/value_monitor.log; tracking en data/web_watch/precio_demo.json</code>	[OK] parse_number con formatos mixtos	06 \$7.2, 14 \$5.5	[!]
F-27	Archivar una página: Markdown + PNG + hash	Tarjeta · <code>POST /api/run/27_web_page_archive</code>	<code>27_web_page_archive</code>	<code>extract_and_save_markdown→browser.extract_content; capture_evidence→browser.capture_page; write_metadata→filesystem.write_json</code>	<code>browser.extract.py::render_markdown, content_hash; browser_capture.py::capture_page</code>	<code>output/*.md, output/screenshots/*.png, output/reports/*.json</code>	[OK] render_markdown cubierto	05 \$1.8	[!]

1.3 Familia sistema — 10 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-05	Snapshot de CPU/RAM/disco con reglas	Tarjeta · <code>POST /api/run/05_system_healthcheck</code>	<code>05_system_healthcheck</code>	<code>take_snapshot→system.snapshot_system; evaluate_snapshot→rules.evaluate; write_snapshot→filesystem.write_json</code>	<code>actions/system.py::snapshot_system; actions/rules.py::evaluate_rules</code>	<code>output/reports/system_health*.json + histórico</code>	[OK] test_actions_basic.py + smoke_test.py	06 \$14	[OK]
F-06	Top 10 procesos con alertas por umbral	Tarjeta · <code>POST /api/run/06_process_watchdog</code>	<code>06_process_watchdog</code>	<code>top_processes→system.top_processes; watch_processes→system.watch_processes; write_report→filesystem.write_json</code>	<code>actions/system.py::top_processes, watch_processes</code>	<code>output/reports/process_watchdog*.json + histórico</code>	[OK] test_actions_basic.py + smoke_test.py	05 \$1.4	[OK]
F-08	Bloquear la estación de trabajo	Tarjeta · <code>POST /api/run/08_windows_lock_workstation</code>	<code>08_windows_lock_workstation</code>	<code>lock_workstation→ui.hotkey ["win", "l"]</code>	<code>actions/ui.py::hotkey</code>	Histórico	[!] Solo en dry_run	05 \$1.5	[manual]
F-10	Abrir el Explorador en una ruta	Tarjeta · <code>POST /api/run/10_explorer_open_path</code>	<code>10_explorer_open_path</code>	<code>open_explorer→ui.launch_process</code>	<code>actions/ui.py::launch_process con shlex y shell=False</code>	Histórico	[!] test_security_hardenimg.py verifica que shell=True se rechaza	11 \$2.2	[manual]
F-11	Abrir una sección de Configuración de Windows	Tarjeta · <code>POST /api/run/11_settings_open_section</code>	<code>11_settings_open_section</code>	<code>open_setting→ui.open_url con URIs - settings:</code>	<code>actions/ui.py::open_url + webbrowser</code>	Histórico	[NO]	09 \$6	[manual]
F-13	Nota rápida en el Bloc de notas	Tarjeta · <code>POST /api/run/13_notepad_quick_note</code>	<code>13_notepad_quick_note</code>	<code>ui.launch_process; system.wait_seconds; ui.type_text</code>	<code>actions/ui.py::launch_process, type_text</code>	Histórico	[!] Solo en dry_run	05 \$1.5	[manual]
F-14	Ejecutar un comando desde el diálogo Ejecutar	Tarjeta · <code>POST /api/run/14_run_dialog_command</code>	<code>14_run_dialog_command</code>	<code>ui.hotkey; system.wait_seconds; ui.type_text; ui.hotkey (4 pasos)</code>	<code>actions/ui.py::hotkey, type_text</code>	Histórico	[!] Solo en dry_run	05 \$1.5	[manual]
F-15	Capturar el portapapeles a JSON	Tarjeta · <code>POST /api/run/15_clipboard_capture</code>	<code>15_clipboard_capture</code>	<code>read_clipboard→system.read_clipboard; save_clipboard→filesystem.write_json</code>	<code>actions/system.py::read_clipboard + pyperclip</code>	<code>output/reports/clipboard*.json + runs.context_json</code>	[!] La rama de degradación sí, el contenido real no	08 \$10.1, 11 \$6	[manual]

F-18	Auditoría Power Shell con allowlist	Tarjeta · <code>POST /api/run/18_powershell_audit</code>	<code>18_powershell_audit</code>	<code>run_audit_command→system.run_powershell; save_audit_report→filesystem.write_json</code>	<code>actions/system.py::run_powershell,_PSS_DEFAULT_ALLOWLIST</code>	<code>output/reports/*.json</code> + histórico	[OK] <code>test_security_harden.py</code> cubre los dos controles	11 \$5	[!]
F-20	Alternar el silencio del audio	Tarjeta · <code>POST /api/run/20_volume_mute_toggle</code>	<code>20_volume_mute_toggle</code>	<code>toggle_mute→ui.hotkey ["volumemute"]</code>	<code>actions/ui.py::hotkey</code>	Histórico	[!] Solo en <code>dry_run</code>	05 \$1.5	[manual]

1.4 Familias **filesystem** y **documentos** — 2 casos

#	Funcionalidad	Interfaz	Manifest	Pasos → acciones	Módulo · función	Persistencia	Prueba	Doc	Estado
F-03	Inventario de una carpeta	Tarjeta · <code>Alt+3</code> (pide ruta)	<code>03_folder_inventory</code>	<code>scan_folder→filesystem.list_directory; classify_inventory→filesystem.classify_file_inventory; write_inventory→filesystem.write_json</code>	<code>actions/filesystem.py::list_directory, classify_file_inventory, write_json</code>	<code>output/reports/folder_inventory_*.json</code> + histórico	[OK] <code>test_actions_basic.py + smoke_tests.py</code>	06 \$14	[OK]
F-04	Pipeline documental de una carpeta de entrada	Tarjeta · <code>POST /api/run/04_document_drop_pipeline</code>	<code>04_document_drop_pipeline</code>	<code>scan_dropbox; classify_dropbox; summarize_texts→filesystem.summarize_text_folder; write_pipeline_report</code>	<code>actions/filesystem.py::summarize_text_folder</code>	<code>output/reports/document_drop_pipeline_*.json</code> + histórico	[OK] <code>test_actions_basic.py</code>	08 \$4	[OK]

2. Capacidades del motor

#	Capacidad	Interfaz	Módulo · función	Persistencia	Prueba	Doc	Estado
M-01	Cargar un manifest y convertirlo en objetos	Interna	<code>engine/loader.py::FlowLoader.load_manifest→engine/models.py</code>	—	[OK] <code>test_loader.py</code> — 100 %	06 \$5.1	[OK]
M-02	Resolver el contexto por precedencia	Interna	<code>engine/loader.py::load_context</code>	Lee <code>configs/, context.user.json, context.example.json</code>	[OK] <code>test_loader.py</code>	10 \$3.1	[OK]
M-03	Sustituir placeholders	Interna	<code>engine/template.py::render_value, flatten_context</code>	<code>steps.params_json</code>	[OK] <code>test_template.py</code> — 100 %	06 \$2	[OK]
M-04	Evaluar condiciones (13 operadores)	<code>when</code> de pasos y transiciones	<code>engine/conditions.py::evaluate_condition, matches</code>	<code>steps.status = skipped</code>	[OK] <code>test_conditions.py</code> — 90 %	06 \$4	[OK]
M-05	Ejecutar el bucle con transiciones	Interna	<code>engine/orchestrator.py::run, resolve_transition, _default_next</code>	<code>runs, steps, events, state/, logs/</code>	[OK] <code>test_orchestrator.py</code> — 87 %	06 \$1	[OK]
M-06	Reintentar y recuperarse por transición	<code>retries</code> y <code>on: "failure"</code>	<code>engine/orchestrator.py::run</code>	<code>steps.attempt, evento step_recovered</code>	[OK] <code>test_orchestrator.py</code>	06 \$1.2	[OK]
M-07	Aplicar la política de sandbox	<code>allowed_actions, allowed_paths, required_secrets, max_runtime_seconds</code>	<code>engine/sandbox.py::SandboxPolicy</code>	<code>runs.context_json → policy, error_json.kind</code>	[OK] <code>test_sandbox.py</code> — 95 %	11 \$4	[OK]
M-08	Resolver acciones de forma perezosa	Interna	<code>engine/action_registry.py::LazyActionRegistry.get</code>	—	[OK] <code>test_action_registry.py</code> — 84 %	03 \$4	[OK]

M-09	Cargar acciones de terceros por entry point	pyproject.toml de l tercero	LazyActionRegistry._maybe_load_entry_points	—	[!] Parcial	09 \$7	[!]
M-10	Detectar los archivos de salida	Interna	engine/introspection.py::extract_existing_paths	runs.outputs_json	[OK] 90 %	06 \$12	[OK]
M-11	Persistir estado, pasos y eventos	Interna	engine/database.py, logger.py, state_store.py	Las 7 tablas + logs/ + state/	[OK] test_orchestrator.py, test_metrics.py	07	[OK]
M-12	Separar raíz de lectura y de escritura	AUTOMA_ROOT, AUTOMA_DATA_ROOT	engine/paths.py::root_dir, data_dir	Determina dónde vive todo	[OK] 3 pruebas en test_actions_basic.py — 86 %	10 \$5	[OK]
M-13	Resolver secretos (entorno, después archivo)	secrets/secrets.json, entorno	engine/secrets.py::get_secret, set_secret, list_secret_names	secrets/secrets.json sin cifrar	[OK] test_secrets_and_notify.py — 73 %	11 \$6	[OK]
M-14	Calcular la próxima ejecución de un cron	Panel · pestaña Programadas	engine/cron.py::next_after, parse_cron	schedules.next_run_at	[OK] test_cron.py — 87 %	06 \$9	[!]
M-15	Disparar tareas programadas	Automático cada 2 s	engine/scheduler.py::SchedulerService	schedules.last_run_at, run_locks	[!] 51–76 %, variable	06 \$10	[!]
M-16	Evitar ejecución doble por lock	Automático · solo scheduler	engine/database.py::acquire_run_lock, release_run_lock	run_locks	[OK] test_run_locks.py	15 · R-07	[!]
M-17	Validar manifests contra el JSON Schema	automa-validate, CI	engine/manifest_schema.py; scripts/validate_project.py	—	[OK] test_manifest_schema.py — 53 %	06 \$14	[OK]
M-18	Agregar métricas	GET /metrics, /api/metrics	engine/metrics.py::overview, by_flow, prometheus_text	Consulta runs y steps	[OK] test_metrics.py — 100 %	09 \$3	[OK]
M-19	Listar y ejecutar desde el CLI	automa list, automa run, automa scheduler	engine/runner.py::main, build_parser	Igual que el panel	[NO] 0 % — y ahí vive el defecto R-06	15 · R-06	[NO]

3. Capacidades del panel

#	Capacidad	Ruta	Módulo · función	Persistencia	Prueba	Doc	Estado
P-01	Panel de 3 pestañas	GET /	app/server.py::render_home, _flow_card_run_tab, _flow_card_schedule_tab, _history_row	Lee flows y runs	[!] test_panel_endpoints.py	09 \$1.1	[!]
P-02	Ejecutar un flow de forma asíncrona	POST /api/run/<folder>	app/server.py::do_POST + threading.Thread	runs, steps, events	[!]	09 \$2	[!]
P-03	Ejecutar de forma síncrona desde formulario	POST /run?flow=<folder>	app/server.py::do_POST	Igual	[!]	09 \$1.2	[!]
P-04	Polling del estado paso a paso	GET /api/runs/<run_id>/status	app/server.py::run_status_payload	Lee runs, steps, manifest	[!]	09 \$2	[!]
P-05	Detalle de una corrida	GET /run/<flow_id>/<run_id>	render_run_detail, _smart_summary, _output_thumb	Lee runs, steps, events	[NO]	08 \$6	[NO]
P-06	Resumen legible por flow	Dentro de P-05	app/server.py::smart_summary	—	[NO]	15 · R-15	[NO] Solo 6 de 27 flows
P-07	Editar el contexto de un flow	GET/POST /flow/<flow_id>/<config>/config	render_flow_config; database.set_flow_config	flow_configs + configfigs/<f>.json	[!]	10 \$3	[!]
P-08	Programar un flow	POST /flow/<flow_id>/schedule	database.set_schedule; engine/cron.py	schedules	[OK] Vía test_cron.py y test_metrics.py	09 \$2	[!]

P-09	Histórico por flow	GET /flow/<f>/history	render_flow_history	Lee runs	[NO]	09 \$1.1	[NO]
P-10	Dashboard de métricas	GET /metrics/dashboard	render_metrics_dashboard	Lee agregados	[NO]	09 \$3	[NO]
P-11	Servir archivos de salida	GET /file?path=...	app/server.py::do_GET, 4 controles	Lee output/	[OK] test_security_harden.py	11 \$2.3	[OK]
P-12	Webhook entrante	POST /api/hook/<folder>	_check_webhook_token; ejecución síncrona	runs, steps, events	[!]	09 \$2	[!]
P-13	Guardar un envío de formulario	POST /api/form/submit	app/server.py::do_POST	output/reports/form_submission_panel_*.json	[NO]	09 \$1.2	[NO]
P-14	Autorizar mutaciones	Todos los POST salvo el webhook	_authorize_mutation, _check_token con hmac.compare_digest	—	[OK] test_panel_endpoints.py, test_security_harden.py	11 \$3.2	[OK]
P-15	Validar el slug de la URL	Las 7 rutas con <folder>	_safe_folder, _FOLDER_RE	—	[OK] test_security_harden.py	11 \$2.2	[OK]
P-16	Marcar un flow como preview	Manifest o archivo .disabled	app/server.py::is_preview	Responde 409	[NO]	10 \$5	[NO]
P-17	Ventana nativa de escritorio	automa-desktop	app/desktop.py::launch, _wait_for_server	—	[!] 48 %	02 \$6	[!]
P-18	Sonda de vida	GET /healthz	app/server.py::do_GET	—	[OK] test_panel_endpoints.py	09 \$1.1	[OK]

4. Reglas de negocio verificables

#	Regla	Dónde se implementa	Cómo comprobarla	Prueba	Estado
RN-01	Un paso cuyo when no se cumple se registra como skipped con attempt: 0 , no se omite	engine/orchestrator.py	Ver la fila en steps con status='skipped'	[OK] test_orchestrator.py	[OK]
RN-02	Un flow se recupera de un fallo solo si la transición on: "failure" apunta a un paso distinto del siguiente por defecto	engine/orchestrator.py	Evento step_recovered en events	[OK] test_orchestrator.py	[OK]
RN-03	La primera corrida de un flow con tracking nunca reporta cambio	browser_extract.py	first_run: true en el resultado	[OK] test_browser_extract.py	[OK]
RN-04	El flow 07 no repite registro hasta agotar los 100; entonces reinicia y lo declara	browser_form.py	cycle_resetted en el resultado	[NO]	[NO]
RN-05	rules.evaluate se detiene en la primera regla que coincide	actions/rules.py	evaluated_rule y evaluations	[OK] test_actions_basic.py	[OK]
RN-06	El crawl respeta robots.txt , y si no lo puede leer permite y lo declara	browser_extract.py	robots_checked_hosts y robots_blocked	[OK] test_browser_extract.py	[OK]

RN-07	Un enlace caído durante el crawl no aborta el recorrido	<code>browser_extract.py: crawl_errors</code>	<code>CrawlErrors</code> en el reporte	[OK] <code>test_browser_extract.py</code>	[OK]
RN-08	<code>system.run_powershellactions/system.py: r</code> rechaza encadenamiento y verbos fuera de la allowlist	<code>actions/system.py: r</code>	<code>ValueError</code> con el motivo	[OK] <code>test_security_hardening.py</code>	[OK]
RN-09	<code>ui.launch_process</code> prohíbe <code>shell=True</code>	<code>actions/ui.py: launch</code>	<code>ValueError</code> explicativo	[OK] <code>test_security_hardening.py</code>	[OK]
RN-10	El sandbox bloquea la corrida entera, no solo el paso	<code>engine/orchestrator.py: runs</code>	<code>status='failed', error_json.kind='san</code>	[OK] <code>test_sandbox.py,</code> <code>test_orchestrator.py</code>	[OK]
RN-11	Toda cota aplicada se declara en el resultado	<code>check_urls,</code> <code>extract_content,</code> <code>read_clipboard,</code> <code>run_powershell,</code> <code>crawl_pages</code>	Campos <code>*_truncated</code>	[OK] <code>test_browser_extract.py</code>	[!] Dos excepciones: <code>summarize_text_folder</code> y <code>apply_tracking</code> (ver 15 · R-24)
RN-12	Un flow sin <code>allowed_actions</code> corre con política permisiva, y el histórico lo registra	<code>engine/sandbox.py: is</code> <code>summary</code>	<code>runs.context_json</code> → <code>policy.permissive</code>	[OK] <code>test_sandbox.py</code>	[OK]
RN-13	El motor corta a <code>max_steps_per_run</code> (200) para evitar ciclos infinitos	<code>engine/orchestrator.py: FlowExecutionError</code>	con el mensaje	[!]	[!]
RN-14	Una acción de terceros nunca puede sobrescribir una interna	<code>LazyActionRegistry._</code> con <code>setdefault</code>	<code>Register an entry point</code> point homónimo	[NO]	[NO]
RN-15	Un placeholder exacto devuelve el valor con su tipo original	<code>engine/template.py: r</code>	<code>"{{ headless }}"</code> llega como <code>False</code> , no <code>"False"</code>	[OK] <code>test_template.py</code>	[OK]
RN-16	Solo se consideran salidas los archivos bajo <code>output/</code>	<code>engine/introspection.py: runs</code>	<code>outputs_json:ing</code>	[OK]	[OK]

5. De la persistencia hacia atrás

Camino inverso: dado un archivo o una tabla, quién lo produce.

Almacén	Escribe	Desde el flow	Se lee en
<code>runs</code>	<code>engine/database.py: upsert_run</code> ← <code>Orchestrator._persist</code>	Todos	<code>catalog.list_runs,</code> <code>metrics.*</code> , panel
<code>steps</code>	<code>insert_step</code> ← <code>Orchestrator.run</code>	Todos	<code>_run_status_payload,</code> <code>render_run_detail</code> , <code>metrics</code>
<code>events</code>	<code>insert_event</code> ← <code>JsonLogger.write</code>	Todos	<code>render_run_detail</code>

<code>flows</code>	<code>sync_flows ← import de app.server, runner</code>	—	Panel
<code>flow_configs</code>	<code>set_flow_config ← POST /flow/<f>/config</code>	—	<code>get_flow_by_folder</code>
<code>schedules</code>	<code>set_schedule, mark_schedule_run</code>	—	<code>scheduler.run_pending_once</code>
<code>run_locks</code>	<code>acquire_run_lock ← solo scheduler._run_job</code>	—	<code>RUNBOOK.md</code>
<code>logs/*.jsonl</code>	<code>JsonLogger.write</code>	Todos	Manual
<code>state/*.json</code>	<code>StateStore.save ← _persist</code>	Todos	Manual
<code>output/screenshots/*.png</code>	<code>screen.capture_*, browser.capture_page</code>	01, 02, 09, 12, 16, 17, 19, 21, 27	<code>GET /file</code>
<code>output/reports/*.json</code>	<code>filesystem.write_json</code>	16 de 27 flows	<code>GET /file</code> , panel
<code>output/reports/*.csv</code>	<code>browser_extract.save_tables_csv</code>	25	<code>GET /file</code>
<code>output/*.md</code>	<code>browser_extract.render_markdown</code>	27	<code>GET /file</code>
<code>output/notifications/*.log</code>	<code>notify.send con backend: "file"</code>	23, 26	Manual
<code>data/seeds/.used_indices.json</code>	<code>browser_form._save_used_ids</code>	07	El propio flow, corrida siguiente
<code>data/web_watch/*.json</code>	<code>browser_extract.apply_tracking</code>	23, 26	El propio flow, corrida siguiente
<code>configs/<folder>.json</code>	<code>set_flow_config</code>	—	<code>FlowLoader.load_context</code>
<code>secrets/secrets.json</code>	<code>engine.secrets.set_secret</code>	—	<code>get_secret</code>

6. Resumen de cobertura de trazabilidad

Categoría	Total	[OK] Verificado	[!] Parcial	[NO] Sin cobertura	[manual] Requiere validación manual
Casos operativos (F)	27	4	8	0	15
Capacidades del motor (M)	19	13	5	1	0
Capacidades del panel (P)	18	4	8	6	0
Reglas de negocio (RN)	16	11	3	2	0
Total	80	32	24	9	15

Lectura del cuadro:

- **El motor está bien cubierto:** 13 de 19 capacidades verificadas por pruebas automáticas. Es el activo mejor protegido del proyecto.
- **Los 15 casos marcados [manual] no son un defecto de las pruebas:** dependen de un escritorio Windows interactivo, de Chromium o del binario `tesseract`. La CI no puede ejecutarlos. La mitigación existente —`dry_run` en las acciones de UI— es

la correcta.

- **Los 9 sin cobertura son huecos reales y abordables:** `engine/runner.py` (donde vive el defecto R-06), seis capacidades de renderizado del panel, y las reglas RN-04 y RN-14. La propuesta priorizada está en [12 §10](#).

Documentos relacionados: [04 · Mapa del código](#) · [05 · Referencia técnica](#) · [07 · Base de datos](#) · [12 · Pruebas y calidad](#) · [15 · Riesgos y deuda técnica](#)